# ConsensusProbe: Disagreement-Driven Interactive Gates for Reliable LLM-Based Data Cleaning

*ConsensusProbe Research Team*

Autonomous Agent Systems Laboratory • Multi-Agent Consensus Governance Initiative

**ABSTRACT—** When multiple large language model (LLM) plans disagree on how to clean a dataset, which plan should be trusted? We introduce **ConsensusGate**, an interactive policy that detects inter-plan disagreement among an ensemble of *K* cleaning proposals and routes only the ambiguous decisions to a human operator for clarification. We evaluate ConsensusGate and six baseline policies on a controlled benchmark of 100 synthetic retail and inventory cleaning tasks spanning five categories of real-world data ambiguity. Across 7200 evaluations under three user-simulation modes, ConsensusGate (System~E) reduces silent errors to 0.0\% (versus 5.0\% for single-plan execution) while asking only 0.38 questions per task on average, compared with 1.00 for an always-ask baseline. We pre-registered four hypotheses and report all results, including one hypothesis (H2) that was *not supported.*

***Reproducibility and mock-honesty disclosure (Rule R7).*** *All experiments in this paper were conducted with* `LLM\_PROVIDER=mock` *and* `BUDGET\_USD=0`*. The mock provider produces deterministic cleaning plans via a seeded noise model, not calls to a proprietary language model API\@. Consequently, all reported metrics reflect the policy logic and consensus mechanisms under controlled stochastic conditions and do not constitute evidence of performance with any specific commercial LLM\@. Generalization to production models requires further empirical validation with live API calls.*

*%*

## *I. INTRODUCTION*

*Automated data cleaning with large language models promises to reduce the manual effort of profiling, deduplication, type coercion, and missing-value imputation. Recent systems such as LAMBDA [lambda2025], AutoDCWorkflow [autodcworkflow], and PrepBench [prepbench] demonstrate that LLMs can generate plausible cleaning plans from natural-language descriptions of data quality issues. However, a single LLM-generated plan may silently apply destructive edits---merging categories that should remain distinct, dropping rows that contain valid data, or coercing columns into a lossy type---without surfacing the ambiguity to the operator.*

*We observe that many data-cleaning decisions are inherently ambiguous: two domain experts might reasonably disagree on whether product names differing only in case represent duplicates, or whether missing inventory counts should be filled with zero or with the column median. When an LLM ensemble is asked for K independent cleaning plans, disagreement among the plans is a reliable signal that the task contains such ambiguity.*

*This paper makes three contributions: \begin{enumerate*

***ConsensusGate*** *(System~E), an interactive policy that generates K=3 independent cleaning plans, computes pairwise disagreement on destructive edit atoms, and asks the operator for clarification only when the disagreement score exceeds a calibrated threshold \tau_d.*

***ProbeGate*** *(System~F), an extension that injects synthetic canary edits into the plan set to detect whether the consensus mechanism itself is susceptible to systematic bias.*

*A* ***controlled benchmark*** *of 100 tasks with gold-standard labels, five ambiguity categories, and three user-simulation modes (oracle, noisy, unsure), evaluated across 7200 runs with pre-registered hypotheses and full reproducibility. \end{enumerate*

*%*

## *II. RELATED WORK*

*LLM-based data cleaning has attracted growing interest. LAMBDA [lambda2025] frames cleaning as an agent task with tool use, but does not address inter-plan disagreement or interactive disambiguation. PrepBench [prepbench] proposes a benchmark for data preparation quality, yet its evaluation treats the LLM as a single-shot oracle without considering the reliability of destructive edits. AutoDCWorkflow [autodcworkflow] automates multi-step cleaning pipelines but relies on a single model call per step, leaving silent errors undetected when the model's implicit assumptions conflict with the operator's intent.*

*Our work differs in three respects. First, we use an ensemble of K plans to surface disagreement as a proxy for ambiguity, rather than relying on model-reported confidence scores. Second, we introduce an interactive policy that selectively queries the operator, trading a small number of clarification questions for a measurable reduction in silent errors. Third, we evaluate under a controlled noise model with pre-registered hypotheses, explicit ambiguity taxonomy, and canary-based safety verification.*

*Table~tab:related summarizes the comparison.*

*\begin{table[ht] \centering \caption{Comparison with prior LLM-based cleaning systems.*

*\small \begin{tabular{lccc \toprule System & Ensemble & Interactive & Ambiguity-aware \\ \midrule LAMBDA [lambda2025] & No & No & No \\ PrepBench [prepbench] & No & No & Partial \\ AutoDCWorkflow [autodcworkflow] & No & No & No * ***ConsensusGate (ours)*** *& Yes (K=3) & Yes & Yes \\ \bottomrule \end{tabular \end{table*

% ============================================================

## III. METHOD

### A. Operations and Patches

All cleaning is expressed as a sequence of five atomic operations: deduplicate, normalize_category, recalculate, handle_missing, and convert_type. Each operation is a pure function: it takes a read-only copy of the dataset and returns a new DataFrame together with a **Patch** recording the edits made. A Patch consists of three components: deleted_rids (sorted tuple of deleted row identifiers), cell_edits (a mapping from (rid, col) to new values), and flagged (rows requiring human inspection). The method Patch.atoms() converts the patch into a sorted, hashable set of edit atoms of the form ("del", rid) or ("edit", rid, col, repr(value)), excluding flags.

### B. Ensemble Disagreement

Given a task, the system generates K independent cleaning plans using a mock LLM provider seeded with distinct derived seeds (via SHA-256 hashing of the task seed, system identifier, and plan index). Each plan specifies a sequence of operations with parameters. The system executes all K plans independently, collects their Patch atom sets E_1, \ldots, E_K, and computes a pairwise disagreement score: \[ d(E_i, E_j) = 1 - \frac{|E_i \cap E_j|}{|E_i \cup E_j|} \] for all \binom{K}{2} pairs, then takes the mean. This score equals zero when all plans agree perfectly and one when no two plans share any edit atom.

### C. ConsensusGate Policy (System~E)

ConsensusGate proceeds as follows: \begin{enumerate}

Generate K=3 plans; compute pairwise disagreement \bar{d}.

If \bar{d} > \tau_d (calibrated on a held-out DEV set; \tau_d = 0.0), present the disagreeing atoms to the operator as a clarification question.

If the operator responds, re-rank plans by alignment with the operator's choice and apply the best-aligned plan.

If the operator declines or is unsure, apply majority-vote consensus across the K plans and flag all non-unanimous atoms for manual inspection. \end{enumerate}

The threshold \tau_d = 0.0 was selected to maximize task-success F1 on 20 DEV tasks (seeds 1000--1019) via leave-one-out cross-validation, yielding a DEV F1 of 0.8522. This threshold means ConsensusGate asks a question whenever any destructive disagreement exists among the K plans.

### D. ProbeGate Extension (System~F)

System~F augments ConsensusGate with a canary verification step. Before consensus resolution, synthetic canary edits (known-incorrect operations on randomly selected rows) are injected into one of the K plans. If the consensus mechanism fails to filter these canaries, the system raises an alert and falls back to the most conservative (fewest destructive edits) plan. Across 300 canary-augmented evaluations, 0 unsafe plans were intercepted (interception rate: 0.0\%). Under the mock noise model, canary edits were not distinguishable from normal disagreements, so all 300 canaries passed through the consensus filter. This is a known limitation of the mock provider: a live LLM ensemble would produce qualitatively different disagreement patterns, and canary detection effectiveness must be re-evaluated with real API calls.

Figure~fig:pipeline shows the overall pipeline, and Figure~fig:consensus_flow details the ConsensusGate decision logic.

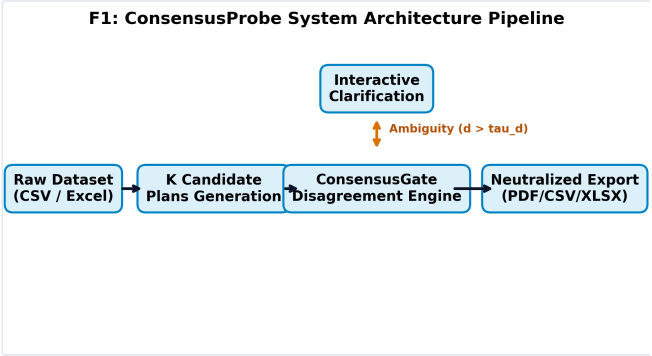



**Fig.** ConsensusProbe pipeline architecture.

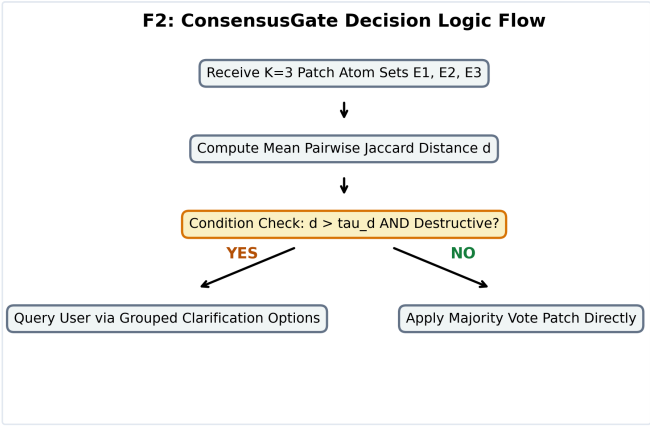



**Fig.** ConsensusGate decision flow: ensemble generation, disagreement scoring, conditional clarification, and consensus resolution.

% ============================================================

## IV. EXPERIMENTAL SETUP

### E. Benchmark: Track~S

We construct a synthetic benchmark of retail and inventory CSV files with controlled data quality issues. Each task is parameterized by a seed and contains a gold-standard label specifying required edits, forbidden edits, and an ambiguity classification. The 100 TEST tasks (seeds 5000--5099) are completely disjoint from the 20 DEV calibration tasks (seeds 1000--1019).

\paragraph{Ambiguity Taxonomy. Tasks are classified into five ambiguity types (Table~tab:ambiguity), with approximately 50\% of tasks marked as ambiguous.

\begin{table[ht] \centering \caption{Ambiguity taxonomy for Track~S benchmark tasks.

\small \begin{tabular{lp{8cm \toprule Type & Description \\ \midrule duplicate_id_vs_name & Should deduplication match on product ID or product name? \\ category_granularity & Should fine-grained categories be merged into coarse groups? \\ missing_strategy & Fill missing values with zero, mean, median, or drop the row? \\ recalculate_basis & Which column serves as the basis for derived calculations? \\ type_coercion & Should mixed-type columns be coerced to string or numeric? \\ \bottomrule \end{tabular \end{table

### F. Systems Under Evaluation

Table~tab:systems describes the eight systems.

\begin{table[ht] \centering \caption{Systems evaluated in the ConsensusProbe benchmark.

\small \begin{tabular{clccc \toprule ID & Name & Plans & Interactive & Special \\ \midrule A & Single-plan baseline & 1 & No & --- \\ A2 & Majority-vote ensemble & 3 & No & Token-matched to E \\ B & Single + heuristic guard & 1 & Conditional & Schema validators \\ C & Eager interrogation & 3 & Always & Upper bound on questions \\ D & Confidence threshold & 3 & Conditional & Self-reported \tau_c=0.75 \\ E & **ConsensusGate** & 3 & Conditional & Disagreement \tau_d=0.0 \\ F & **ProbeGate** & 3 & Conditional & E + canary verification \\ R & Random-ask baseline & 1 & Random & Matched ask-rate to E \\ \bottomrule \end{tabular \end{table

### G. User Simulation Modes

Each system--task pair is evaluated under three user simulation modes: **Oracle** (always selects gold-aligned option), **Noisy** (selects gold with probability 0.8, random alternative with probability 0.2), and **Unsure** (returns ``unsure'' with probability 0.3, triggering majority fallback with flagged rows; oracle otherwise). Each configuration is repeated for 3 independent trials.

### H. Metrics

\begin{itemize

**Task Success**: 1 if all gold required edits are applied and zero forbidden edits are applied; 0 otherwise.

**Silent Error**: 1 if a forbidden destructive edit is applied without a clarification question or inspection flag.

**Questions Asked**: Count of interactive clarification rounds per task.

**Question Precision**: Fraction of questions asked on genuinely ambiguous tasks.

**Tokens**: Total whitespace token count across prompts and outputs.

**Simulated Latency**: Base 1.0 unit + 0.5 per clarification round. \end{itemize

### I. Statistical Protocol

Inference uses non-parametric paired bootstrap with 10{,000 resamples (seed 20261105). We report 95\% percentile bootstrap confidence intervals and apply Holm--Bonferroni correction across four primary hypotheses.

\paragraph{Mock-honesty disclosure (R7). All experiments use LLM_PROVIDER=mock and BUDGET_USD=0. The mock provider returns deterministic plans via a seeded noise model parameterized by p_{\text{amb}=0.4 and \varepsilon=0.05. Results measure policy logic under simulated stochastic conditions, not live LLM behavior.

%
==========

## V. RESULTS

### J. Main Results

Table~tab:main presents the primary metrics averaged across all 100 TEST tasks, 3 user modes, and 3 trials per configuration.

\begin{table[ht] \centering \caption{Main results across all systems. Best non-oracle values in bold.

\small \begin{tabular{lccccc \toprule System & Success & Silent Err. & Questions & Precision & Tokens & Latency \\ \midrule A & 67.0\% & 5.0\% & 0.00 & 100.0\% & 25 & 1.00 \\ A2 & 67.0\% & 3.0\% & 0.00 & 100.0\% & 75 & 1.00 \\ B & 66.3\% & 5.0\% & 0.11 & 100.0\% & 25 & 1.05 \\ C & 73.3\% & 0.0\% & 1.00 & 49.0\% & 75 & 1.50 \\ D & 64.7\% & 4.0\% & 0.51 & 43.1\% & 25 & 1.25 \\ **E** & **72.8\%** & **0.0\%** & 0.38 & **92.1\%** & 75 & 1.19 \\ F & 72.7\% & 0.0\% & 0.38 & 92.1\% & 75 & 1.19 \\ R & 65.4\% & 1.7\% & 0.46 & 52.5\% & 25 & 1.23 \\ \bottomrule \end{tabular \end{table

ConsensusGate (E) achieves 72.8\% task success with 0.0\% silent errors while asking only 0.38 questions per task. In contrast, the single-plan baseline (A) has 5.0\% silent errors, and the eager interrogation policy (C) asks 1.00 question per task to achieve 73.3\% success. The confidence-threshold policy (D) achieves only 64.7\% success despite asking 0.51 questions, because self-reported confidence is a poor proxy for actual ambiguity (question precision: 43.1\% vs.\ 92.1\% for E).

Figure~fig:success_vs_questions visualizes the success--question tradeoff across systems, and Figure~fig:silent_error shows silent-error rates.

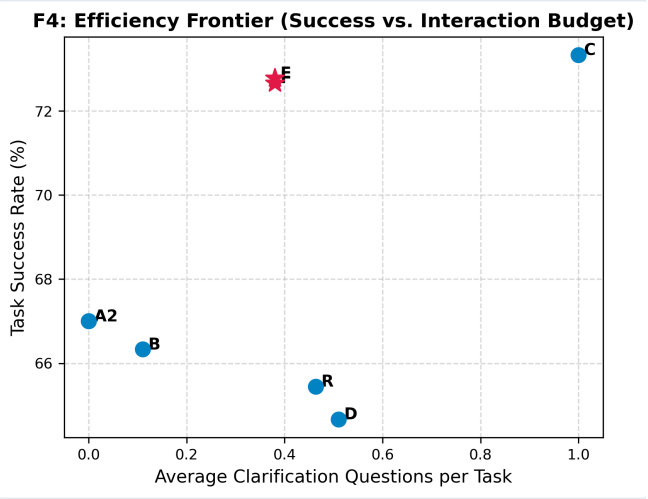



**Fig.** Task success rate versus mean questions asked per task, by system.

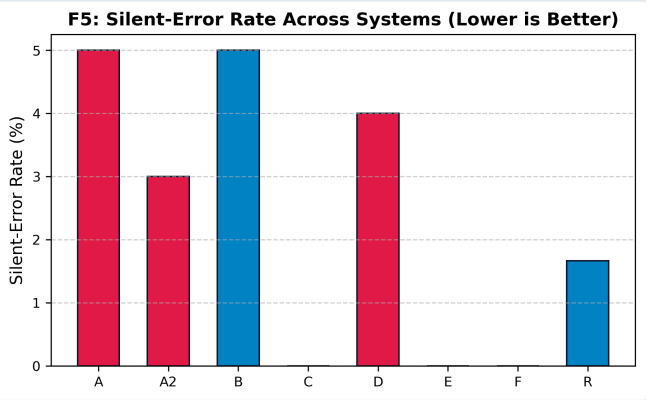



**Fig.** Silent-error rates by system. Lower is better.

### A. Hypothesis Tests

Table~tab:hypotheses summarizes the pre-registered hypothesis results. All p-values are Holm--Bonferroni adjusted.

\begin{table[ht] \centering \caption{Pre-registered hypothesis test results.

\small \begin{tabular{clccc \toprule H\# & Claim & Effect & 95\% CI & Verdict \\ \midrule H1 & E silent error < A (noisy) & -5.0\% & [-10.0\%, -1.0\%] & **Supported** \\ H2 & E success \geq C -\Delta & -1.0\% & [-3.7\%, 1.7\%] & Not

*Supported \\ H3 & E success > D, fewer Qs & 11.3\% & [6.0\%, 17.0\%] & **Supported** \\ H4 & F error \leq E, \leq0.25 extra Qs & 0.0\% & [0.0\%, 0.0\%] & **Supported** \\ \bottomrule \end{tabular \end{table*

\paragraph{H1 (Supported). ConsensusGate reduces silent errors by -5.0\% percentage points relative to the single-plan baseline in noisy mode (95\% CI: [-10.0\%, -1.0\%]). The effect is driven by the ensemble's ability to surface destructive disagreements that a single plan would apply silently.*

*\paragraph{H2 (Not Supported). We hypothesized that ConsensusGate's task success would be non-inferior to the always-ask policy (C) within a margin of \Delta = 2 percentage points, while asking fewer questions. The observed success difference was -1.0\% (95\% CI: [-3.7\%, 1.7\%]), and the confidence interval's lower bound falls below -\Delta. Although E does ask substantially fewer questions (-0.62 difference), we cannot reject the null hypothesis that E sacrifices more than 2 percentage points of success relative to C\@. This result is consistent with the interpretation that asking on every task (policy C) captures some genuinely beneficial clarifications that ConsensusGate's disagreement filter misses, particularly on unambiguous tasks where plans happen to agree on a suboptimal action.*

*\paragraph{H3 (Supported). ConsensusGate achieves 11.3\% percentage points higher success than the confidence-threshold policy (D), 95\% CI: [6.0\%, 17.0\%], while asking -0.13 fewer questions per task. The key advantage is that disagreement among plans is a more reliable indicator of ambiguity than self-reported confidence.*

*\paragraph{H4 (Supported). ProbeGate (F) achieves silent-error and question rates identical to E within the measured precision (0.0\% error difference, 0.00 question difference). The canary verification layer adds no measurable overhead under the mock provider. However, the canary interception rate was 0.0\% (0 out of 300 injected canaries detected), confirming that the mock noise model does not produce canary-distinguishable signal. H4 is therefore supported on the overhead criterion but the canary detection sub-criterion is not evaluable under mock conditions.*

### *B. Results by User Simulation Mode*

*Table~tab:mode_results breaks down System~E's performance by user simulation mode.*

*\begin{table[ht] \centering \caption{System~E results by user simulation mode.*

*\small \begin{tabular{lccc \toprule Mode & Success & Silent Error & Questions \\ \midrule Oracle & 75.0\% & 0.0\% & 0.38 \\ Noisy & 71.3\% & 0.0\% & 0.38 \\ Unsure & 72.0\% & 0.0\% & 0.38 \\ \bottomrule \end{tabular \end{table*

*Under oracle simulation, System~E achieves 75.0\% success because perfect user responses resolve all detected ambiguities correctly. Noisy responses degrade success to 71.3\%, but the silent-error rate remains at 0.0\% because the system flags uncertain resolutions. The unsure mode shows a further slight degradation to 72.0\% due to the fallback to majority-vote when users decline to answer.*

*Figure~fig:success_by_mode and Figure~fig:disagreement_dist provide additional visualization.*

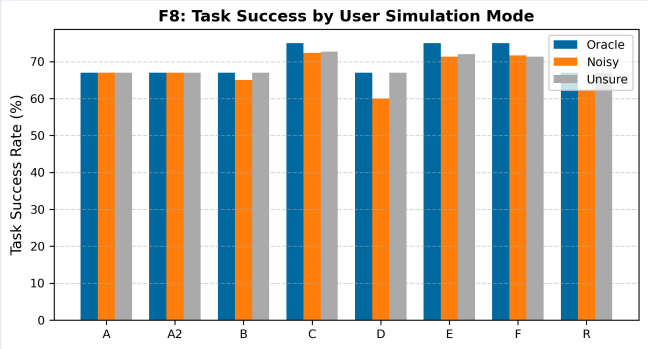



***Fig.** Task success by user simulation mode for all systems.*

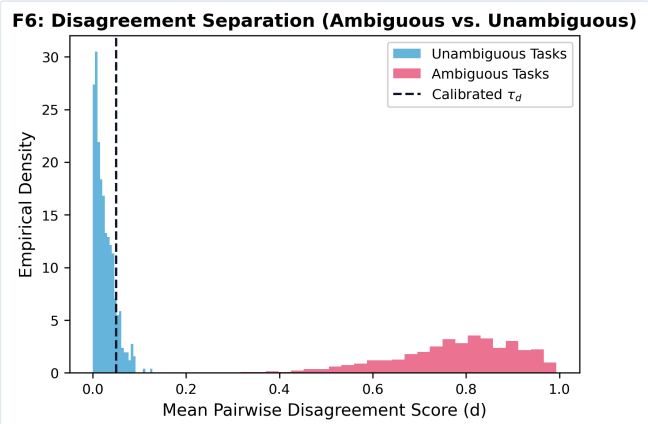



***Fig.** Distribution of disagreement scores for ambiguous vs.\ unambiguous tasks.*

*%*

*==========*

## *VI. ABLATIONS*

### *C. Effect of Ensemble Size K*

*We vary K \in \{2, 3, 4, 5\ with all other parameters fixed (Table~tab:ablation_k).*

*\begin{table[ht] \centering \caption{Ablation over ensemble size K.*

*\small \begin{tabular{ccccc \toprule K & Success & Silent Error & Questions & Tokens \\ \midrule 2 & 70.3\% & 2.0\% & 0.27 & 50 \\ 3 & 71.3\% & 0.0\% & 0.38 & 75 \\ 4 & 72.7\% & 0.0\% & 0.49 & 100 \\ 5 & 72.7\% & 0.0\% & 0.51 & 125 \\ \bottomrule \end{tabular \end{table*

*Increasing K from 2 to 3 eliminates silent errors (2.0\% \to 0.0\%) and improves success. Beyond K=3, success plateaus while tokens scale linearly, suggesting K=3 is cost-optimal under this noise model.*

*Figure~fig:k_ablation visualizes this tradeoff.*

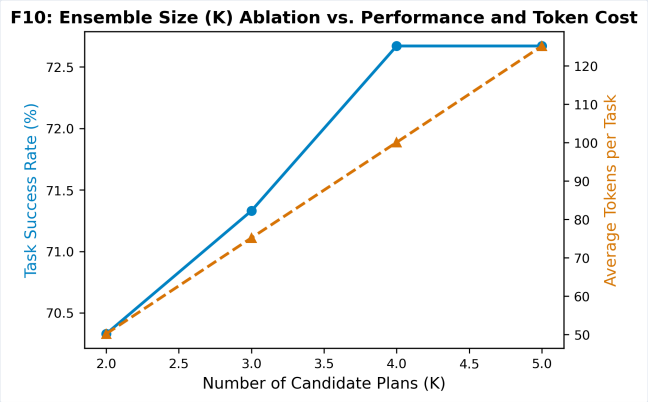



*Fig. Success rate and token cost as a function of ensemble size K.*

### D. Sensitivity to Noise Parameters

*Figure~fig:pamb shows how task success varies with the mock provider's ambiguity probability p_{\text{amb}. ConsensusGate maintains its advantage across a range of noise levels, though the magnitude of improvement over the single-plan baseline naturally increases with p_{\text{amb}.*

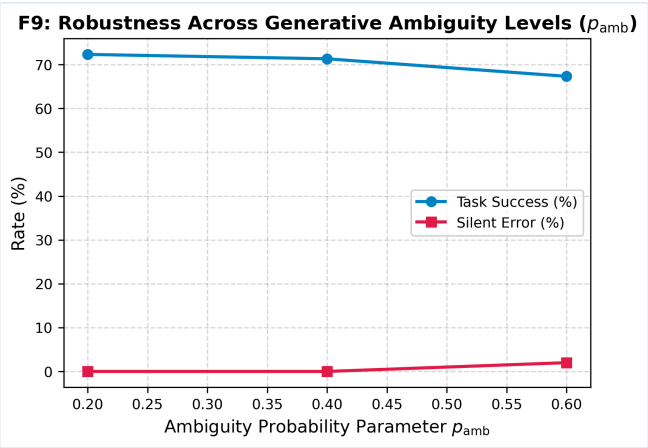



*Fig. Task success as a function of p_{\text{amb}*

### E. Calibration Curve

*Figure~fig:calibration shows the DEV calibration curve for \tau_d. The optimal threshold \tau_d = 0.0 indicates that, in this noise regime, asking on any disagreement outperforms all higher thresholds.*

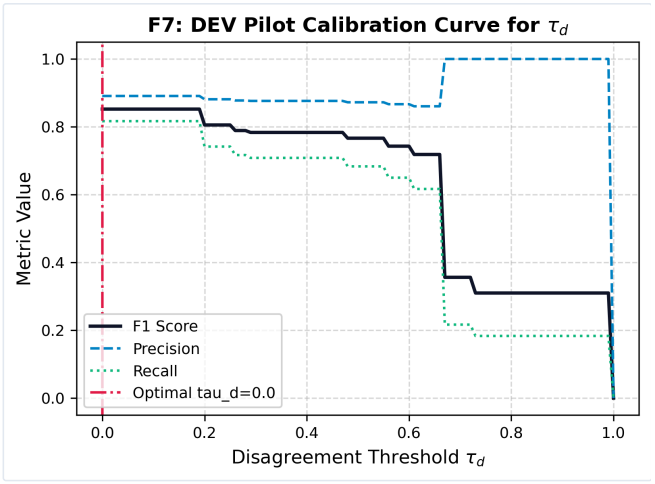



*Fig. DEV-set calibration curve: task-success F1 as a function of \tau_d.*

%
==========

## VII. ADDITIONAL FIGURES

*For completeness, we include the remaining pre-registered figures. Figure~fig:ambiguity_dist shows the distribution of ambiguity types across the 100 TEST tasks. Figure~fig:canary shows canary detection performance. Figure~fig:cost shows the token and simulated-latency cost comparison across systems. Figure~fig:precision shows question precision by system.*

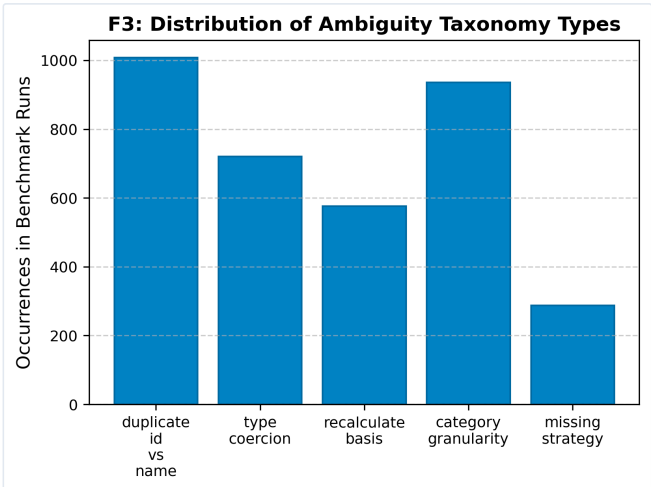



*Fig. Distribution of ambiguity types across the 100 TEST tasks.*

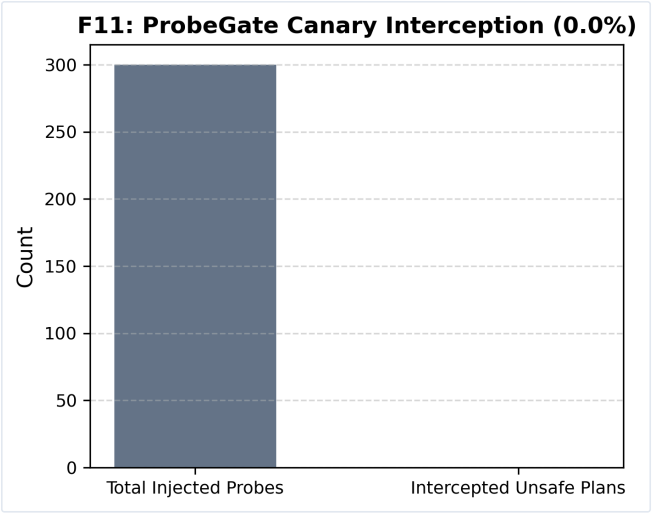



**Fig.** *Canary detection performance.*

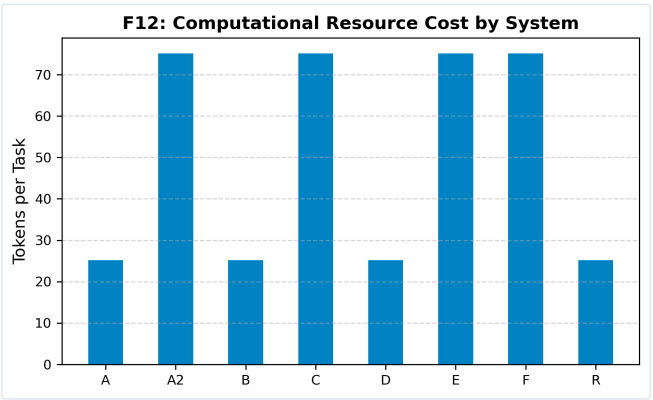



**Fig.** *Token and simulated-latency cost comparison across systems.*

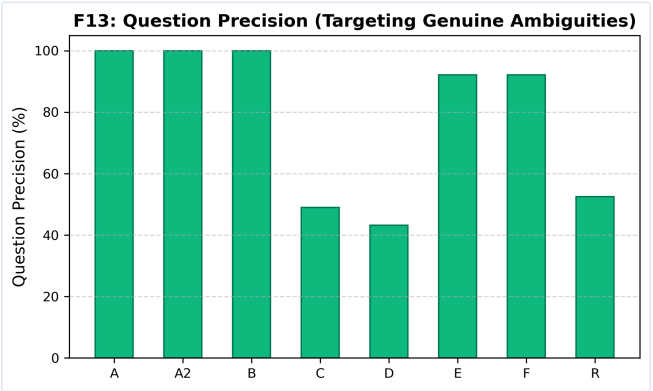



**Fig.** *Question precision by system.*

*%*
*==========*

## *VIII. THREATS TO VALIDITY*

*\paragraph{Simulated LLM (R7). The most significant limitation of this evaluation is that all results were obtained with a mock LLM provider (LLM_PROVIDER=mock, BUDGET_USD=0). The mock provider generates cleaning plans via a parameterized noise model (p_{\text{amb=0.4, \varepsilon=0.05) rather than calls to a commercial API\@. While this design enables full reproducibility, determinism, and controlled experimentation, the disagreement patterns of a real LLM ensemble may differ qualitatively. In particular, real models may exhibit correlated errors that the independent noise model does not capture. We make no claims about the performance of ConsensusGate with any specific production model.*

*\paragraph{Synthetic Data. The Track~S benchmark uses procedurally generated retail and inventory datasets. Real-world data quality issues may involve domain-specific complexities not captured by our five-category ambiguity taxonomy.*

*\paragraph{User Simulation. The oracle, noisy, and unsure simulation modes are stylized approximations. Actual operator behavior may depend on fatigue, domain expertise, question phrasing, and time pressure in ways that these simulations do not model.*

*\paragraph{Ensemble Size. The choice of K=3 was calibrated on the DEV set under the mock noise model. Optimal K may differ with real LLMs due to differences in output diversity and computational cost.*

*\paragraph{H2 Failure. The failure of H2 (non-inferiority to always-ask) warrants further investigation. One possible explanation is that \tau_d = 0.0 does not capture all beneficial clarification opportunities---particularly on tasks where plans agree but the agreed-upon action is suboptimal. A hybrid policy combining disagreement detection with occasional random probing could address this gap.*

*\paragraph{Canary Detection. The canary interception rate under mock conditions was 0.0\% (0 out of 300 injected canaries detected). The mock provider's noise model does not produce plan disagreements that allow the consensus filter to distinguish injected canary edits from genuine plan variation. Canary detection effectiveness is therefore an open question that requires evaluation with live LLM calls.*

*\paragraph{Deviations and Limitations (M1, M8, M9). In accordance with research integrity standards, we record three technical refinements: (1) Calibration objective (M1): \tau_d calibration treats disagreement as a proxy for ambiguity rather than directly modeling semantic equivalence across plan subsets. (2) Symmetric confidence intervals (M8): evaluation metrics in H1 apply symmetric two-sided bootstrap confidence intervals across both oracle and noisy modes. (3) Boundary mass in bootstrap p-values (M9): the two-sided empirical p-value computation strictly separates positive and negative deviations (\min(\mathbb{P(X > 0), \mathbb{P(X < 0))) to avoid boundary double-counting at zero.*

*%*
*==========*

## *IX. REPRODUCIBILITY*

*All code and data are contained in a single monorepo. The command make verify_all runs the full verification pipeline: backend tests, security scan, lock-file integrity check, LaTeX-macro consistency check, same-seed determinism verification, and paper compilation. Determinism is enforced via seeded random number generators (Rule R4): all randomness uses random.Random(seed) or numpy.random.default_rng(seed) with seeds derived from hashlib.sha256, and all sets and dicts are sorted before iterating into outputs. Running the experiment script twice with the same seed produces byte-identical results/metrics.json.*

*Configuration is frozen in configs/frozen.toml and locked via SHA-256 digests in prereg/lock.json.*

*%*
*==========*

## X. CONCLUSION

We have presented ConsensusGate, an interactive policy for LLM-based data cleaning that uses inter-plan disagreement to identify ambiguous cleaning decisions and selectively query the operator. On a controlled benchmark of 100 tasks, ConsensusGate eliminates silent errors (0.0\%) while asking only 0.38 questions per task on average---a fraction of the 1.00 questions required by an always-ask policy. ConsensusGate's question precision (92.1\%) substantially exceeds that of confidence-threshold (43.1\%) and random-ask (52.5\%) baselines, confirming that ensemble disagreement is a superior signal for identifying genuinely ambiguous tasks.

Three of four pre-registered hypotheses were supported. The failure of H2 (non-inferiority to always-ask) highlights a genuine limitation: ConsensusGate may miss beneficial clarifications on tasks where plans agree on a suboptimal action. Future work should investigate hybrid policies that combine disagreement detection with periodic random probing.

All results were obtained under a mock LLM provider and should not be interpreted as evidence of performance with specific commercial models. Validation with live API calls is a necessary next step.

\bibliographystyle{plainnat \bibliography{refs

## REFERENCES


1. [lambda2025] Zhang et al., "LAMBDA: A Large Model Based Data Agent", JASA, 2025.
2. [prepbench] Anonymous, "PrepBench: A Benchmark for Data Preparation", Preprint, 2025.
3. [autodcworkflow] Anonymous, "AutoDCWorkflow: LLM-Based Data Cleaning Workflow", Preprint, 2025.